

CI/CD

Version: 1.0

Purpose

This document defines the Continuous Integration and Continuous Delivery strategy of Trading Platform Pro and its primary application, the Trading Cockpit.

The CI/CD pipeline provides automated quality assurance for:

- source code
- architecture boundaries
- automated tests
- trading-critical workflows
- documentation sources
- documentation generation
- future release artifacts

CI/CD shall reduce integration risk without introducing trading behaviour.

Objectives

The CI/CD pipeline shall provide:

- deterministic quality checks
- reproducible dependency installation
- automated source validation
- automated formatting verification
- automated tests
- architecture boundary validation
- trading workflow regression protection
- PAPER and LIVE safety validation
- documentation source validation
- documentation generation validation
- fast developer feedback
- transparent failure reporting

CI/CD Principles

The pipeline shall be:

- deterministic
- reproducible
- explicit
- automated
- fail-fast where appropriate
- transparent
- independent from LIVE trading
- safe for repeated execution

CI/CD shall never:

- submit LIVE orders
- require LIVE broker connectivity
- depend on a LIVE trading account
- mutate production trading state
- silently bypass failed quality gates

Continuous Integration

Every relevant repository change shall be automatically verified.

The target CI sequence is:

```
```text
```

Checkout Repository

↓

Setup Python

↓

Install Dependencies

↓

Source Quality Checks

↓

Architecture Boundary Checks

↓

Unit Tests

↓

Integration Tests

↓

Trading Workflow Regression Tests

↓

Documentation Source Validation

↓

Documentation Generation Validation

↓

Publish CI Results

```
```
```

Pipeline failures shall block integration where the failed step is a required quality gate.

Repository Checkout

The pipeline shall use a clean repository checkout.

CI shall not depend on:

- developer-local files
- local virtual environments
- untracked configuration
- local database state
- manually generated artifacts

Required repository state shall be reproducible from source control.

Python Environment

CI shall use an explicitly supported Python version.

The Python version shall be defined in the repository or CI workflow.

Local development and CI should use compatible Python versions.

Python version changes require:

- dependency compatibility review
- automated test execution
- documentation updates where required

Dependency Installation

Dependencies shall be installed from repository-managed dependency definitions.

Current dependency files may include:

```
```text
```

requirements.txt

requirements-dev.txt

requirements-docs.txt

```
```
```

Production, development and documentation dependencies shall remain distinguishable.
CI shall not depend on globally installed developer packages.
Dependency installation shall occur inside the CI environment.

Source Quality Checks

Source quality checks shall include:

```
```bash
ruff check .
```
```

Formatting verification shall use:

```
```bash
ruff format --check .
```
```

CI shall verify formatting.

CI shall not automatically rewrite source files during validation.

Local development may use:

```
```bash
ruff format .
```
```

to apply formatting before commit.

Architecture Boundary Checks

Architecture boundaries are part of the system quality model.

Automated architecture checks should be introduced where practical for rules such as:

- Domain does not depend on Infrastructure
- Domain does not depend on Presentation
- Application does not depend on Presentation
- Presentation does not access concrete broker adapters directly
- SQLAlchemy models do not leak into Domain
- PySide6 types do not leak into Domain
- provider-specific broker models do not leak into Domain

Architecture checks shall validate documented dependency rules.

Do not create architecture checks for hypothetical boundaries that are not part of the documented architecture.

Unit Tests

Unit tests shall run in CI.

Example:

```
```bash
pytest tests/unit -q
```
```

Unit tests shall remain:

- deterministic
- isolated
- fast
- independent from LIVE services

Domain and Application business rules require focused unit tests.

Integration Tests

Integration tests shall validate implemented technical boundaries.

Examples may include:

- repository implementations
 - SQLite persistence
 - configuration loading
 - dependency registration
 - broker adapter translation
 - market data adapter translation
 - runtime service coordination
- Integration tests shall not require LIVE trading.

External systems shall use:

- fake adapters
- controlled test environments
- explicit PAPER integration where separately approved

Trading Workflow Regression Tests

Trading-critical workflows require explicit regression protection.

Regression tests shall cover implemented workflows such as:

- Trading Candidate lifecycle
- Trading Decision lifecycle
- Order validation
- Order submission workflow
- Duplicate submission prevention
- Broker acknowledgement distinction
- Broker rejection
- Partial fill handling
- Filled order handling
- Position lifecycle
- Reconciliation discrepancy detection

Tests shall preserve distinctions between:

``text

Submission Requested

Transmitted

Acknowledged

Rejected

Partially Filled

Filled

``

A successful adapter call shall not be tested as broker acknowledgement unless broker-derived acknowledgement state exists.

Duplicate Submission Safety

Duplicate order submission risk requires automated regression tests.

Tests shall verify where implemented:

- stable order identity
- submission identity
- duplicate detection
- duplicate blocking
- repeated callback handling
- repeated broker state messages
- restart or recovery behaviour

CI shall fail when duplicate-prevention regression tests fail.

PAPER and LIVE Safety Tests

PAPER and LIVE separation requires automated tests.

Tests shall verify:

- PAPER configuration remains PAPER
 - LIVE configuration requires explicit selection
 - LIVE is not activated through fallback
 - PAPER does not silently use LIVE broker configuration
 - execution environment is available to application workflows
 - execution environment is visible in operational state
 - invalid LIVE configuration fails startup validation
- CI itself shall not activate LIVE trading.

Broker Tests

Broker integration tests shall focus on adapter and workflow behaviour.

Tests may simulate:

- connection
- disconnection
- reconnection
- acknowledgement
- rejection
- execution
- partial fill
- repeated broker messages
- provider errors

Broker tests shall not blindly retry order submission.

Reconnection tests shall remain separate from order submission tests.

Market Data Tests

Market data tests may verify:

- connection state translation
- quote translation
- source timestamp preservation
- unavailable values
- stale data state
- subscription activation
- subscription failure
- subscription closure

Market data tests shall preserve the distinction between:

- current
- stale
- unavailable
- disconnected

Runtime Tests

Runtime tests shall verify implemented lifecycle behaviour.

Examples:

- deterministic startup order
- startup failure handling
- runtime state transitions
- degraded state
- task ownership
- cancellation
- broker connection failure
- market data connection failure
- subscription cleanup

- controlled shutdown
 - repeated shutdown requests
- Runtime tests shall not require LIVE trading.

Reconciliation Tests

Reconciliation workflows require regression protection.

Tests shall verify where implemented:

- local state loading
- broker state loading
- discrepancy detection
- discrepancy classification
- no-discrepancy result
- action-required state
- failure handling

Monitoring and logging shall not repair reconciliation state.

Reconciliation state changes require explicit Application workflow ownership.

Documentation Source Validation

Markdown files under:

```
```text
docs/
```
```

are the documentation source of truth.

CI documentation validation shall operate on Markdown source files.

Generated DOCX and PDF files shall not be used as authoritative documentation input.

Documentation validation may verify:

- Markdown file readability
- required document structure
- invalid internal references where supported
- documentation generation compatibility

Generated Documentation

Generated documentation resides under:

```
```text
docs/generated/
```
```

Current generated formats:

```
```text
docs/generated/docx/
docs/generated/pdf/
```
```

Generated files are derived artifacts.

Generated DOCX and PDF files shall not be edited manually.

Documentation Generation Validation

CI shall validate that the documentation generation script completes successfully.

Command:

```
```bash
python scripts/generate_docs.py
```
```

The validation succeeds when:

- the script exits successfully
- required Markdown source files can be processed
- DOCX generation completes
- PDF generation completes

CI shall not perform content validation by re-reading generated DOCX or PDF files.

This follows the project decision to validate Markdown source documents as the authoritative content.

Generated File Validation Boundary

Generated DOCX and PDF files are output artifacts.

CI shall not:

- parse generated DOCX files for content comparison
- parse generated PDF files for content comparison
- use generated files as documentation source
- compare formatting internals of DOCX or PDF files

CI may verify that expected output files were created where generation validation requires it.

The Markdown source remains authoritative.

Documentation Generation Environment

Documentation generation dependencies shall be installed from:

```
```text
requirements-docs.txt
```
```

The CI documentation job shall use the project environment.

CI shall not depend on globally installed document-generation tools.

Required external executables shall be documented explicitly if introduced.

Documentation Failure

Documentation generation failure shall fail the documentation validation job.

Failure output shall identify:

- source Markdown file where available
- target format
- generation stage
- exception context

CI shall not silently skip failed documents.

GitHub Actions

CI is implemented using GitHub Actions.

Workflow files reside under:

```
```text
.github/workflows/
```
```

Workflows shall remain explicit and focused.

Potential workflow separation may include:

```
```text
quality.yml
tests.yml
docs.yml
```
```

Workflow separation shall only be introduced where it improves maintainability.

GitHub Actions Safety

GitHub Actions shall not have unrestricted access to LIVE trading credentials.
CI workflows shall not connect to LIVE broker execution services.
Repository secrets shall only be introduced for defined CI requirements.
Trading credentials shall not be added to CI solely for integration testing.

Branch Strategy

Primary branch:

```
```text
main
```
```

Optional feature branches:

```
```text
feature/
```
```

Bug fixes:

```
```text
bugfix/
```
```

Additional branch types require a defined workflow requirement.
Avoid complex branching strategies without a concrete collaboration need.

Pull Request Quality Gates

Before integration into `main`, required quality gates shall pass.
Current or target gates include:

- Ruff check
 - Ruff format verification
 - Unit tests
 - Integration tests where implemented
 - Trading workflow regression tests where implemented
 - Architecture boundary checks where implemented
 - Documentation source validation
 - Documentation generation validation
- A required failed gate shall block integration.

Local Verification

Before pushing, developers should run the checks relevant to their change.
Standard source verification:

```
```bash
ruff check .
ruff format --check .
pytest tests/unit -q
```
```

Documentation changes should additionally run:

```
```bash
python scripts/generate_docs.py
```
```

Integration tests should be run when affected infrastructure boundaries change.

Local Formatting

To apply Ruff formatting locally:

```
```bash
ruff format .
```
```

After formatting, verify:

```
```bash
ruff format --check .
```
```

CI verifies formatting but does not modify source files.

Test Selection

CI may separate test categories for execution efficiency.

Example:

```
```text
Unit Tests
Integration Tests
Trading Regression Tests
Documentation Tests
```
```

Test separation shall remain explicit.

Critical trading regression tests shall not be omitted solely to reduce CI duration.

Test Failures

Failed tests shall fail the corresponding CI job.

Flaky tests shall not be ignored indefinitely.

A flaky test shall be:

1. identified
 2. investigated
 3. stabilized
 4. documented where temporary quarantine is unavoidable
- Trading-critical tests shall not be silently skipped.

Quality Gate Ownership

Each quality gate shall have a defined purpose.

Examples:

```
```text
Ruff
→ Source quality
Unit Tests
→ Isolated business and application behaviour
Integration Tests
→ Technical boundary integration
Trading Regression Tests
→ Business-critical workflow protection
Documentation Validation
→ Documentation source integrity
Documentation Generation
→ Derived artifact generation
```
```

Avoid duplicate quality gates that verify the same behaviour without additional value.

Continuous Delivery

Continuous Delivery shall evolve with concrete release requirements.

Potential future delivery stages include:

- version validation
- build artifacts
- release packaging
- release notes
- signed artifacts
- deployment validation

Delivery automation shall remain independent from application business logic.

Do not implement automated deployment before a concrete deployment target exists.

Release Artifacts

Future release artifacts may include:

- application packages
- installers
- documentation packages

Generated documentation may be included in release artifacts.

Release artifact structure shall be defined when packaging requirements are known.

Release Versioning

Release versioning shall use one documented versioning strategy.

Version changes shall remain synchronized where version information is exposed.

Automated versioning shall not be introduced until release workflow ownership is defined.

Release Readiness

Before release verify:

- required CI quality gates passed
- trading-critical regression tests passed
- PAPER and LIVE safety tests passed
- documentation source updated
- documentation generation succeeded
- version information updated where required
- changelog updated
- release notes prepared
- known critical issues resolved

Only release software that satisfies defined release criteria.

Rollback Strategy

Delivery workflows shall define rollback behaviour before automated deployment is introduced.

Rollback strategy depends on the future deployment model.

Potential rollback concerns include:

- application version
 - configuration compatibility
 - database schema compatibility
 - persisted business state
- Rollback shall not silently rewrite trading history.

Dependency Auditing

Dependency auditing is a planned quality capability.

Future CI may include:

- known vulnerability checks
- dependency metadata validation
- unsupported dependency detection

Dependency auditing shall be introduced when the selected tooling and response policy are defined.

Do not document dependency auditing as active until implemented.

Security Scanning

Security scanning is a planned quality capability.

Potential future checks may include:

- source scanning
- secret scanning
- dependency vulnerability scanning

Security scanning shall be integrated with explicit ownership and failure policy.

LIVE trading credentials shall never be exposed to scanning output.

Coverage Reporting

Coverage reporting may be introduced to support test analysis.

Coverage percentage shall not replace risk-based testing.

High coverage does not prove safe trading workflow behaviour.

Trading-critical workflows require explicit scenario tests.

CI Observability

CI results shall clearly identify:

- failed job
- failed quality gate
- failed command
- relevant test or document
- failure output

CI logs shall not expose secrets.

Failure diagnostics shall remain sufficient for local reproduction.

CI Reproducibility

A failed CI check should be reproducible locally where practical.

Commands documented in CI shall use repository-managed dependencies and paths.

Avoid CI-only hidden validation logic.

Where CI-specific behaviour is required, document the reason.

CI/CD Evolution

CI/CD evolves with implemented product and release requirements.

Before introducing a new pipeline step:

1. Identify the quality or delivery risk.
2. Identify the owning workflow.
3. Define the validation behaviour.
4. Define failure policy.
5. Evaluate runtime cost.
6. Ensure local reproducibility where practical.
7. Update documentation.

Avoid pipeline complexity created only for hypothetical future scale.

CI/CD Review Checklist

Before introducing or changing CI/CD verify:

- quality or delivery requirement identified
- pipeline step has one clear purpose
- execution is deterministic
- execution is reproducible
- LIVE trading is not required
- LIVE credentials are not exposed
- architecture boundaries evaluated
- trading regression impact evaluated
- PAPER and LIVE safety evaluated
- documentation source validation evaluated
- documentation generation evaluated
- generated DOCX/PDF content is not treated as source
- failure policy defined
- local reproduction documented
- runtime cost evaluated
- rollback impact evaluated where relevant
- documentation synchronized

Related Documents

- Product_Vision.md
- Product_Roadmap.md
- Project_Overview.md
- Architecture.md
- Infrastructure.md
- Technical_Specifications.md
- Project_Structure.md
- Configuration.md
- Runtime.md
- Logging.md
- Monitoring.md
- Testing_Strategy.md
- Development_Guidelines.md
- Git_Workflow.md
- AGENTS.md